

A Co-evolutionary Algorithm approach to a University Timetable System

C. K. Chan, H. B. Gooi and M. H. Lim

eckchan@ntu.edu.sg

School of Electrical and Electronic Engineering

Nanyang Technological University

Nanyang Avenue

Singapore

Abstract - This paper describes an automated curriculum timetabling system based on a stochastic search methodology, namely a co-evolutionary algorithm. The application timetable is taken from the undergraduate courses of the School of Electrical and Electronic Engineering (EEE), Nanyang Technological University (NTU). A co-evolutionary algorithm approach is found to be well suited. Practical courses have duration greater than one hour. A schedule can be generated separately and its population, which consists of a set of practical schedules, is termed as the practical population. Lecture and tutorial schedules can also be generated separately. These are of one-hour duration and they are termed collectively as lecture/tutorial schedule. A set of lecture/tutorial schedules could be generated to form the lecture/tutorial population. These two populations use the same set of resources and have constraining effects upon one another. Since the placement of practical courses have a more constraining effect, the schedules in the practical population are first generated and are then used to guide the generation of the set of lecture/tutorial schedules. For every lecture/tutorial schedule generated, it is combined with its corresponding practical schedule to form a combined schedule. The average fitness of all the combined schedules is then computed and used as a measure of the fitness of the practical schedule that drives them. The practical population is then evolved progressively to obtain the best practical schedule. It is then used as a base configuration for the rest of the courses to populate and evolve. The resultant system compares favorably to the current manual system.

I. INTRODUCTION

An educational institution, such as a University, relies heavily on a curriculum timetable to function smoothly. A curriculum timetable is usually a schedule containing a set of teaching activities planned over a week. This timetable is used in NTU over an academic semester, which is 13 weeks long. The planning of such a timetable is not trivial. Some of the courses of EEE in NTU have more than 1000 students. Such courses have to be divided into smaller lecture classes of about 350 each. Hence, the same lecture may have to be repeated over a week. At NTU, the planning of timetables is carried out manually, aided by word-processing, spreadsheet, database and the internally developed facility-booking systems. Commercial timetabling software was once considered but later abandoned as it was felt that the NTU

curriculum is fairly dynamic and such software would become obsolete very fast. In addition, commercial software is complex to use and do not meet the needs of the School directly.

The current approach to planning the timetable was to manually manipulate the previous timetable, taking into account impending changes. However, with the introduction of the modular system and the increasing number of students, such an approach is no longer efficient. Some students are also taking a combination of subjects that would not have been feasible in the past. For example, a third year student, in addition to taking the third year subjects, may also take some second and final year subjects.

Many heuristic algorithms for timetabling have been proposed. They include simulated annealing, constraint logic programming, linear programming and graph coloring heuristic [1,2,3,4]. Heuristic algorithms use the concept of a multi-dimensional search space, which contains the complete solution set. Heuristic techniques are employed to explore the search space for the best solution. The main difficulty faced in this approach is the tendency to be trapped in a local optimal point, which may not be the best.

Researchers are now turning to evolutionary algorithms, or its hybrid, as a powerful method of solving the timetabling problems. Evolutionary algorithms [5,6] are powerful general-purpose optimization technique, which models the principles of evolution. However, the structure of the EEE timetable is not apt for a traditional evolutionary algorithm approach. This arises out of the need to incorporate courses having different duration per class session.

The main objective of this paper is to present the feasibility of automating the timetabling process in EEE by using a co-evolutionary algorithm. There are five types of courses: lecture, tutorial, project, laboratory and design. Project, laboratory and design courses can be further grouped as practical courses. Practical courses, which have duration per class session longer than one hour, are scheduled first on its own without considering lecture and tutorial courses. The set

of the schedules of such practical courses constitutes a practical population. The lecture and tutorial courses, being of one-hour duration per class session, can be scheduled together. The set of these schedules constitutes another population. This population is conveniently termed as the lecture/tutorial population. There is a lot of similarity between these two populations. They share the same set of resources. This has given rise to some constraining effect upon one another. Hence the generation of a lecture/tutorial schedule has to take into account the practical schedule. Because of this interaction, they have to be co-evolved. This paper describes the co-evolution of these two populations.

II. EEE, NTU Curriculum Timetable

In NTU, there are 6 schools or faculties, EEE being the largest. Students join EEE in their second year, after successfully completing a common-curriculum in the first year. Third year students are attached to the industry for practical training in semester two. As such, there is no scheduled teaching for third year students in semester two. Final year subjects are divided into nine option groups where each group has its own prescribed subjects. They must choose 4 subjects with a minimum of 16 academic units. Each academic unit is equivalent to approximately one hour of lecture per week per semester. In addition, they have to study a number of free elective subjects of general nature.

All the students have to study core subjects pertaining to their current year of study. Most of these core subjects are offered in both semesters. Furthermore, students who have failed in some subjects have to retake and pass them within the university specified time frame.

There are three lecture groups for the second year and third year students. A lecture course for a particular year must be conducted for every lecture group. Each of this lecture group is further divided into 12 smaller tutorial subgroups. Similarly, within each lecture group, there are 12 subgroups for each of the laboratory, design and project courses as well. To minimize clashes, a student is only allowed to choose a subgroup belonging to his lecture group.

In planning a timetable, there are two types of constraints that must be considered. They are termed as the hard and soft constraints.

There are two hard constraints that are universal to all timetabling problems. They are:

- a. No student can be in two or more places at any one period;
- b. There must be sufficient seating capacity in the venue for all the registered students scheduled in any one period.

For EEE timetable, there are additional hard constraints to be satisfied. There must be an hour break between two practical sessions. The same lecture course should not be conducted on the same day for the same group of students. Certain specified timeslots are reserved for other purposes. All the lecture, tutorial, project, laboratory and design groups must be scheduled. The duration of the lecture, tutorial, project, laboratory and design groups must be respected in the schedules generated. The start and end time of the classes in each day must be respected in the schedule. Students must be allowed to have lunch within ± 1.5 hours from noon.

Any violation of these hard constraints will result in an infeasible timetable. Evolutionary algorithms operators used in this system are carefully designed to avoid any violation of these hard constraints.

Soft constraints are actually preferences. They can be violated. However, a timetable with many such violations are usually considered as poor in quality. Hence, in most timetable system, the amount of violations is used to determine the fitness value of a schedule. Some of these soft-constraints are: bad time slots, e.g., teaching after office hours; very fragmented lecture sessions in a particular day; venues of consecutive classes are far apart.

III. Co-evolutionary Algorithm

Co-evolution is defined as a change in the genetic composition of one species (or group) in response to a genetic change in another. More generally, the idea of some reciprocal evolutionary change in interacting species is a strict definition of co-evolution.

As discussed earlier, the timetable of EEE consists of the practical schedule and the lecture/tutorial schedule. The placement of practical courses in a practical schedule has a constraining effect upon the placement of lecture and tutorial courses in the lecture/tutorial schedule. Hence, the use of a co-evolutionary algorithm approach to the problem is most appropriate.

The overall flow chart of the co-evolutionary algorithm is shown in Figure 1. The prelude module performs most of the basic set up in the genes, chromosomes and populations of

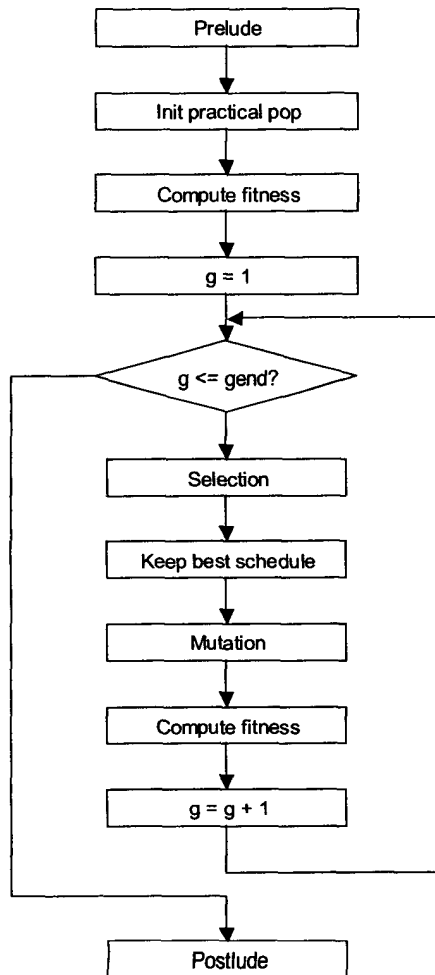


Figure 1 Main flow chart

the system. g is a loop counter for each generational step. $gend$ is the maximum number of generations. The postlude module closes all working files and databases and terminates the program normally.

An initial population of practical schedules is generated by a random placement of all practical courses. Due to the large number of such courses, they are more or less fixed in position in the practical schedule. The only movable entity is the placement of individual student group. With this in mind, mutation of a practical schedule is simply a random mutual swapping of two student groups. The mutation used here is very much akin to the manual planning process where a timetable planner likes to make only minor changes to the

timetable. Although the placement of individual courses in a practical schedule is almost constant, the placement of each practical subgroup is important in directing placement of the other one-hour lecture and tutorial courses. Hence, an optimal placement of student groups in a practical schedule is desirable. This is obtained by using the evolutionary search methodology on the population of practical schedules. One important consideration is then the fitness of each practical schedule. A logical approach is to generate randomly several lecture/tutorial schedules from each practical schedule. Each lecture/tutorial schedule and the corresponding practical schedule are combined into a main schedule. The fitness value of each combined schedule is computed. The average fitness value is then used to express the fitness of the corresponding practical schedule. This is shown in Figure 2.

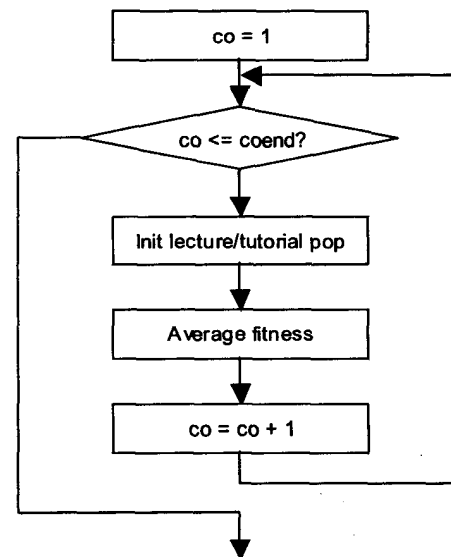


Figure 2 Compute fitness of practical schedule flow chart

In Figure 2, co is a loop counter over the set of schedules in the practical population. $coend$ is the number of schedules in the practical population.

As shown in Figure 2, one of the key operations is to generate a population of lecture/tutorial schedules from every practical schedule. This is usually done randomly. However, the operation is very time-consuming and can fail. Certain practical schedule is very constrained and it is not possible to populate other one-hour courses on it without violating some

serious hard constraints. Such violations will result in an infeasible timetable. Even if there are possible generations of lecture/tutorial schedules, there may be too few of it. This may take many unsuccessful attempts before they are found. This imposes great complexity in the design of the operation. A simple way will be to assign a pre-specified limit on the number of trials on each generation. Upon reaching the limit and there is no generation of any lecture/tutorial schedule, a low fitness value is then assigned to the corresponding practical schedule. A low fitness value for a practical schedule implies that it is very likely to be discarded in the selection module at the next evolutionary step. This approach requires careful consideration of the limit value. If it is set too high, a lot of processing time is wasted. If it is set too low, a good possible practical schedule might be missed. Intuitively, a good practical schedule does not generate many good lecture/tutorial schedules as it is usually very constraining. Hence, a higher limit is desirable.

The fitness value of each combined schedule is computed based on the amount of soft-constraint violations. There are several soft-constraints to be considered in the EEE timetable. One particular important soft-constraint violation is that some courses are conducted outside normal office-hours, which is 5pm for weekday and 1pm for weekend. For EEE, these are timeslots that end at 5:30 pm on weekdays and 1:30 pm on Saturday. These assignments actually represent failures to place all courses on the proper timeslot. Placing courses after office-hours is highly undesirable, as it requires additional resources and manpower. In addition, such timeslots are not popular with students. This will result in an uneven distribution of all the courses. Hence, practical schedules, which allow generation of lecture/tutorial schedules with minimum amount of such violation, are desirable. Therefore, the fitness value of each combined schedule is computed based on the amount of violation of this soft-constraint. There are other soft-constraints, which are not considered here but will be included in future works.

The linear scaling over the fitness values and the roulette selection method are used to select the set of practical schedules for mutation [12]. Just before mutation, the best practical schedule is kept. After mutation, the previously kept best practical schedule replaces the worst practical schedule in the latest population of practical schedules. This is done to preserve the good practical schedules and at the same time allows some probabilistic diversity.

IV. Implementation

Since courses that are of two or three hour duration are considered here, only the second and third year courses are incorporated in testing the system. Most of the final year courses are of one-hour duration and are not related to these lower year courses. However, this option can be considered in the future as the curriculum for the final year courses might be changed.

There are altogether 50 timeslots in a week. Each timeslot is a one-hour slot. Timeslot 1 starts at Monday 8:30 am and ends at Monday 9:30 am. Timeslot 2 starts at Monday 9:30 am and ends at Monday 10:30 am. Timeslot 9, which is considered a bad timeslot, starts at Monday 4:30pm and ends at Monday 5:30pm. Timeslot 50, which is also a bad timeslot, starts at Saturday 12:30 pm and ends at Saturday 1:30 pm. At each timeslot, multiple courses can be conducted as long as no hard-constraints is violated.

The data structure used for the implementation is a linear collection of available resources arranged according to their timeslot in an ascending order. Each of these available resources is termed as a gene and has a type, room capacity and its availability status. In each timeslot, each available resource for that timeslot is coded as a gene in the chromosome. For example, if there are five lecture theatres available in timeslot 1, then five genes for timeslot 1, which have type *lecture* will be coded into the chromosome.

Each collection is like a chromosome in a population. A population consists of a set of these chromosomes. Two populations are needed in this system. They are the practical and lecture/tutorial population. These populations have similar genetic structure. A population of practical schedules consists of a set of such chromosomes with practical courses being properly scheduled in. Similarly, a population of lecture/tutorial schedules consists of a set of such chromosomes with lecture and tutorial courses properly scheduled in.

TABLE I NEEDS AND RESOURCES TABLE

Types of courses	Number of sessions needed	Number of sessions available
Lecture	69	150
Tutorial	152	600
Laboratory	76	80
Project	38	40
Design	38	40

Table 1 shows the needs and resources of the whole second and third year timetable in semester 1. A session is a number of timeslots placed consecutively together. As can be seen in the table, the available sessions for the laboratory, project and design courses are very close to the need. This explains the tight and constraining situations for the practical schedules. Although there are relatively many available sessions for lectures and tutorials, their placements are severely affected by the practical courses placed in parallel with these sessions.

An initial population of five practical schedules is randomly generated. For every generation, an initial population of ten lecture/tutorial schedules is generated on every practical schedule. A schedule from the lecture/tutorial population is combined with its corresponding practical schedule. The fitness of the combined schedule is then computed. This combination of schedules and computation of fitness is repeated over the whole lecture/tutorial population. An average fitness value of these ten fitness values is then computed and is considered being a measure of the fitness of the corresponding practical schedule. This is repeated for every schedule in the practical population. This concludes the computation of one generation. The number of generations is set at 20.

Mutation involves randomly selecting two positions on a particular chromosome of the practical schedule. The distance between the two points is empirically set at 30% of the length of the chromosome. These positions denote the portion of the chromosome whereby mutation can take place. Each of the genes within this portion is considered for swapping with another gene within the same chromosome but not in this portion. Swapping will take place only if no hard-constraints is violated. Hard-constraints violations are clashes in the usage of resources within the chromosome. These have been mentioned earlier.

The whole algorithm is implemented in Microsoft Visual Basic on a PC platform. Microsoft Access is also used to store the timetable and other auxiliary data structures.

V. Results

Figure 3 shows the average fitness value of each population of practical schedule in each evolution step. The result is obtained after 20 generations. The fitness value reflects the average number of unoccupied bad slots. The figure should be as high as possible. The highest possible value for this set of data is 276. As shown in Figure 3, after 20 generations, the average number of unoccupied bad slots saturated at about 230. This reflects the tightness of the

schedule. To ease the situation, either the number of courses offered or the number of student groups has to be reduced.

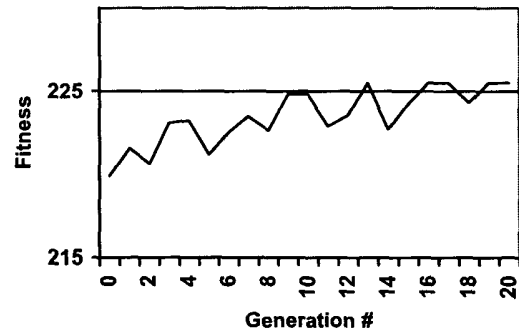


Figure 3 Fitness vs generation

TABLE 2 NO. OF FAILURES VS GENERATION

Generation #	No. of failures	Generation #	No. of failure
1	0	11	3
2	0	12	0
3	1	13	4
4	2	14	2
5	0	15	2
6	0	16	4
7	3	17	4
8	1	18	3
9	4	19	4
10	4	20	4

Table 2 shows the number of infeasible situation whereby a particular practical schedule fails to produce a feasible lecture/tutorial schedule. The number of failures is relative to a population of 5 practical schedules. As the evolution progresses, the practical schedule improves in its fitness. However, the number of failures increases too. This is in a way, expected. A better practical schedule produces fewer lecture/tutorial schedules. In fact, the failures would be higher if the trial limit is set too low. It has been found that the number of trials has to be progressively increased in order to find a set of feasible solutions. Eventually, a maximum of 10 trials has been assigned to each generation of the lecture/tutorial schedule. This has significantly increased the computational time.

VI. Conclusions

A co-evolutionary algorithm approach has been successfully applied to solve a university curriculum timetable, namely that of EEE. The EEE curriculum timetable has a natural intrinsic structure in which separate practical and lecture/tutorial schedule can be obtained. An optimal combined schedule of the two separate schedules, after successive co-evolution between them, has been generated. This is a first step in automating the scheduling of EEE timetable. It will not only relief the tedious task of timetabling, but will provide some form of measurement to a completed timetable. There are many alternative timetables generated for the planner to choose from as compared to the manual system.

Future works will focus on ways to reduce the number of failed attempts and improve on the computation of the fitness value of the practical schedule. The initialization of the lecture/tutorial population module is very time-consuming as compared to the other operations. Hence, future effort will also be focused on reducing the computation time of this module.

References

- [1] E.K. Burke, D. G. Elliman, R. Weare, "Automated Scheduling of University Exams", proceedings of the IEEE Colloquium on Resource Scheduling for Large Scale Planning System, June 1993.
- [2] M.W.Cater, G. Laporte, "Recent Developments in Practical Examination Timetabling", First International Conference in Practice and Theory of Automated Timetabling, pp. 3- 21, Sept 1995.
- [3] M.W.Cater, G. Laporte, "Recent Developments in Practical Examination Timetabling", Second International Conference in Practice and Theory of Automated Timetabling II, pp. 3-19, Aug 1997.
- [4] A. Schaerf, "Tabu Search Techniques for Large High-School Timetabling Problems", proceeding of the 13th National Conference of the American Association for Artificial Intelligence (AAAI-96, 1996),
- [5] E.K. Burke, D.G. Elliman and R.F. Weare, "A Hybrid Genetic Algorithm for Highly Constrained Timetabling Problems", proceedings of the 6th International Conference on Genetic Algorithms pp 605-610, Morgan Kaufmann, San Francisco, CA, USA, (ICGA'95, Pittsburgh, USA, 15th-19th July 1995).
- [6] R. Dawkins "The Selfish Gene", London, U.K.: Oxford Univ. Press, 1976.
- [7] A. Wren, "Scheduling, timetabling and rostering – a special relationship?" The Practice and Theory of Automated Timetabling: Selected Papers from the 1st International Conference, Lecture Notes in Computer Science 1153, pages 46-75, Springer-Verlag, Berlin, 1996.
- [8] E.K.Burke, J.P.Newall, "A Multi Evolutionary Algorithm for the timetable Problem", IEEE Transactions on Evolutionary Computation, Vol. 3, No 1, pp.63-74, April 1999.
- [9] E.K. Burke and A.J. Smith "A Memetic Algorithm for the Maintenance Scheduling Problem", Proceedings of the ICONIP'97 Conference, Dunedin, New Zealand, (published by Springer), pp 469-474, , 24-28 November 1997
- [10] M. W. Varter, G. Laporte, and S.Y. Lee, "Examination timetabling: Algorithmic strategies and application", Dept. Industrial Eng, Univ. Toronto, Working Paper 94-03, Jan. 1995.
- [11] M. R. Garey and D. S. Johnson, "Computers and Intractability: A Guide to the Theory of NP-Completeness", W. H. Freeman, New York, 1979.
- [12] D.E. Goldberg, "Genetic Algorithms in Search, Optimization and Machine Learning", Addison-Wesley, 1989.